

Pokémon Red Benchmark — Jev vs Jev + GPT Planner (Spec)

Overview and hypothesis

The experiment tests whether a fast, non-chat decision model (Jev) needs a slow LLM planner (GPT-6 Sol) to make progress in Pokémon Red.

- **Arm A:** Jev + harness. Jev picks every action with no planner.
- **Arm B:** Jev + GPT-6 Sol + harness. Sol sets a goal every N steps; Jev still makes every final action choice.

Hypothesis: Arm A handles moment-to-moment moves fine but stalls on long-horizon quest logic (e.g. Oak's Parcel, finding HM01 Cut). Arm B should get further per step, at a higher cost per step.

Models

Jev only answers pre-declared questions, so the harness must turn every game moment into a fixed menu of options. GPT-6 Sol is a normal chat LLM that can reason in free text.

	Jev (TypeSafe)	GPT-6 Sol (OpenAI)
Output	Choice (from a fixed list), Score, or Noul (yes/no probability), each with a confidence value	Free text, tool calls, structured output
Limits	32k-token context; a Choice question holds at most 255 options	1.05M-token context; 128k max output
Price (USD per 1M tokens)	\$0.042 input, output free	\$2 input / \$10 output (\$0.20 cached input; Flex tier \$1 / \$5)
Latency	70–500 ms	about 1–5 s median
Access	Early access; waitlist reported. API at api.typesafe.ai/v1/systemone , Python and JS SDKs; also on Vercel AI Gateway	OpenAI API or OpenRouter (openai/gpt-6-sol)
Released	15 Sep 2026	22 Sep 2026

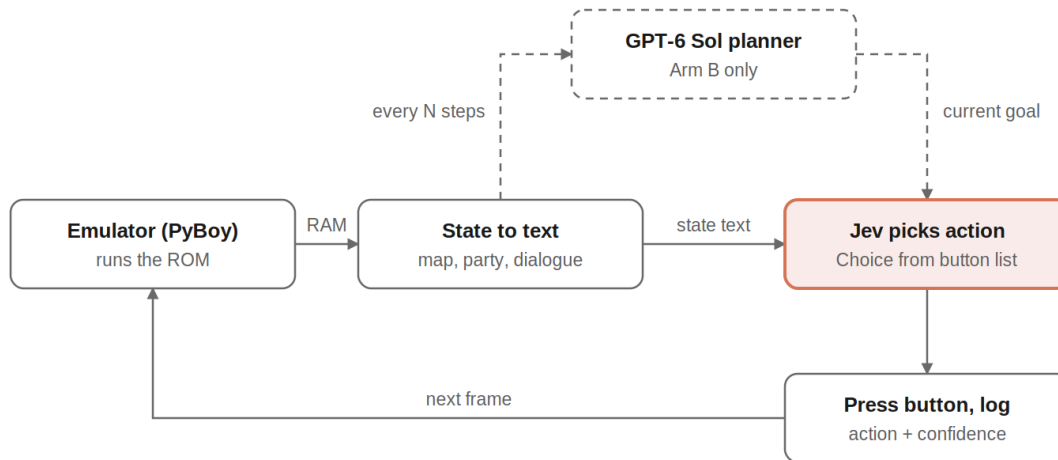
Consequence for this project: Jev never sees a screenshot as far as published docs show. The harness must feed it text state (RAM-parsed position, map, party, dialogue) and a button list.

Sources: [Jev explainer](#), [GEO Toolbox](#) · [GPT-6 Sol](#), [OpenRouter](#)

Architecture

Both arms run the same loop; Arm B adds one dashed branch where Sol writes a goal that Jev sees alongside the state.

Jev makes every move; Sol only resets the goal every N steps



Agent loop · dashed path runs in Arm B only

- **Arm A:** Jev gets state text + a fixed standing goal ("make progress toward the next badge").
- **Arm B:** every N steps Sol reads the state (plus a step history) and returns a short goal, e.g. "walk north to Viridian, deliver parcel". Jev gets state text + that goal and picks the button.
- Jev's confidence is logged every step. Low-confidence streaks are a useful signal to analyse, and a possible later trigger for an early Sol call.

Harness

Decision: NousResearch/pokemon-agent is the base harness. It is a game server rather than an agent: it hands over parsed state through an HTTP API and takes button presses, which suits Jev's typed-question model. Both arms call the same API, so the only difference lives in our own agent code. Borrow maxkskor/pokemon-harness's per-turn logging ideas (observation, action, tokens, cost, screenshot). First task for Claude Code: confirm it installs and has been updated recently.

What the adapted harness must provide:

- **State to text:** position, map ID, facing, party HP/levels, badges, bag, current dialogue or menu text, all from RAM.
- **Action space:** a fixed Choice list (up, down, left, right, A, B, Start, Select, plus optional macros like "walk to exit"). Well under Jev's 255-option cap.
- **Save states and step logging** for every run.
- Everything fits in Jev's 32k-token context, so trim history hard.